

Doubly Linked List – Delete From Position –Space Complexity and Algorithm

Program :

```
void deleteFromPosition(int pos)
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
    if (pos < 1)
    {
        cout << "Invalid position!" << endl;
        return;
    }
    if (pos == 1)
    {
        deleteFromBeginning();
        return;
    }

    Node *temp = head;
    for (int i = 1; i < pos && temp != nullptr; i++)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }

    temp->prev->next = temp->next;
```

```

    if (temp->next != nullptr)
    {
        temp->next->prev = temp->prev;
        temp->next = nullptr;
    }
    temp->prev = nullptr;

    cout << "Deleted " << temp->data << " from position " <<
pos << "." << endl;
    free(temp);
    temp = nullptr;
}

...

case 7:
    cout << "Enter position to delete: ";
    cin >> pos;
    deleteFromPosition(pos);
    break;

```

Space Complexity

Auxiliary Space

Auxiliary space refers to the extra or temporary memory used by an algorithm, not including the space taken by the inputs themselves.

In the deleteFromPosition function, the new memory usage is:

*Node *temp: A single temporary pointer is created to find and hold the address of the node to be deleted. The size of a pointer is constant.*

int i: An integer variable is used as a counter in the for loop. This also occupies a constant amount of space.

The function's main memory operation is free(temp), which deallocates memory. Since the amount of extra memory used does not grow with the size of the linked list, the auxiliary space complexity is $O(1)$.

Auxiliary Space: $O(1)$

Input Space

Input space is the memory used to store the function's inputs. This can be analyzed from two perspectives.

1. Parameter – only Definition

This view considers only the space taken by the function's direct parameters. The function signature is `void deleteFromPosition(int pos)`.

The input is a single integer, `pos`, which occupies a fixed, constant amount of memory.

Under this common definition, the input space is constant.

Input Space (Parameter-only): $O(1)$

2. Full – Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list.

If the linked list contains n nodes before the deletion, the total memory it occupies is proportional to n .

Under this definition, the input space is linear with respect to the size of the list.

Input Space (Full-Data): $O(n)$

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$\text{Space} = \underbrace{O(1)}_{\text{Auxiliary}} + \underbrace{O(1)}_{\text{Input}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, the "space complexity" of a function refers to its auxiliary space. Therefore, the most common and expected answer for this function is $O(1)$. The for loop used for traversal affects the function's time complexity ($O(n)$ in the worst case), but not its space complexity.

Algorithm: Delete from a Given Position (Doubly Linked List)

DDELPOS(START, POS)

1. *[CHECK IF LIST IS EMPTY]*

If START = NULL, then:

Write: "List is empty. Nothing to delete." and EXIT.

[End of If structure.]

2. *[CHECK FOR INVALID POSITION]*

If POS < 1, then:

Write: "Invalid position!" and EXIT.

[End of If structure.]

3. *[CHECK FOR DELETION AT BEGINNING]*

If POS = 1, then:

Call DDELBEG(START) [Use the beginning deletion algorithm].

Translate DDELBEG(START):

DDELBEG(START)

1. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

Write: "List is empty. Nothing to delete." and EXIT.

[End of If structure.]

2. [STORE ADDRESS OF FIRST NODE]

Set PTR := START [PTR is a pointer used to hold the node to be deleted].

3. [ADVANCE START TO THE SECOND NODE]

Set START := NEXTLINK[START].

4. [UPDATE NEW FIRST NODE, IF IT EXISTS]

If START ≠ NULL, then:

Set PREVLINK[START] := NULL (The PREV pointer of the new first node is set to NULL to unlink from previous first node).

[End of If structure.]

5. [OUTPUT THE DELETED NODE]

Write: "DELETED", INFO[PTR], "from the beginning."

6. [DEALLOCATE THE ORIGINAL FIRST NODE]

Set NEXTLINK[PTR] := NULL (The NEXT pointer of the previous first node de – links from the current and new first node)

free(PTR) [Releases the memory of the node pointed to by PTR].

7. [AVOID DANGLING POINTER]

Set PTR := NULL.

8. Exit

EXIT.

[End of If structure.]

4. [TRAVERSE TO THE NODE AT THE TARGET POSITION]

Set PTR := START [PTR is a pointer initialized to the start of the list].

FOR(Set COUNT := 1 ; COUNT < POS AND PTR ≠ NULL ; Set COUNT := COUNT + 1)DO:

SET PTR := NEXTLINK[PTR] .

[END of Loop]

5. *[CHECK IF POSITION IS OUT OF BOUNDS]*

If PTR = NULL, then:

Write: "Position out of bounds!" and EXIT.

[End of If structure.]

6. *[UNLINK THE NODE FROM THE LIST]*

a. Set NEXTLINK[PREVLINK[PTR]] := NEXTLINK[PTR] $\left(\begin{array}{l} \text{The node before PTR now} \\ \text{points to the node after PTR} \end{array} \right)$.

b. If NEXTLINK[PTR] ≠ NULL, then: $\left(\begin{array}{l} \text{This handles cases where the} \\ \text{node is not the last one in the list} \end{array} \right)$.

Set PREVLINK[NEXTLINK[PTR]] := PREVLINK[PTR] $\left(\begin{array}{l} \text{The node after PTR now points its PREV} \\ \text{back to the node before PTR} \end{array} \right)$.

[End of If structure]

7. *[DEALLOCATE THE TARGET NODE]*

Set PREVLINK[PTR] := NULL. [TARGET NODE's PREV pointer got UNLINKED]

Write : " DELETED" ,INFO[PTR], "FROM POSITION", POS .

free(PTR) [Releases the memory of the node pointed to by PTR].

8. *[AVOID DANGLING POINTER]*

Set PTR := NULL .

9. *Exit*
